

Algorithmics	Student information	Date	Number of session
	UO: 307423	19/03	4
	Surname: Miranda Martín	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Mariana Patricia		



Activity 1. [Greedy Algorithms]

Solve the proposed problem creating an Assignment.java class. Execute the class by passing each of the provided file names as a parameter to verify the correct functioning of the implemented algorithm.

Create an AssignmentTimes.java class that, after randomly generating an $n \times n$ productivity matrix, calculates the execution time of the assignment algorithm:

The time should be in milliseconds, with the optimizer enabled, and marked as TLE (Time Limit Exceeded) when it takes longer than one minute.

N	time (ms)
1024	33
2048	39
4096	179
8192	895
16384	4945
32768	30570
65536	TLE
131072	TLE

Create an AssignmentTests.java class that contains 5 test cases (5 methods), one for each input file, to verify that the result is as expected. The test cases must run without requiring any modifications to your source code. Otherwise, the exercise will be graded as a 0.

Reason whether the obtained times align with the complexity of the algorithm (using the formulas).

There is indeed a consistency with the data. As the method implemented has a quadratic theoretical complexity, i.e., $O(n^2)$, by doubling the input each iteration, the execution times should be for times bigger as it grows exponentially.

Algorithmics	Student information	Date	Number of session
	UO: 307423	19/03	4
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Do you think the proposed algorithms (ALGORITHM 1 and ALGORITHM 2) are greedy algorithms? Why?

Yes, the proposed algorithms are Greedy Algorithms.

By definition, an algorithm is a greedy algorithm when it makes the locally optimal choice at each stage with the hope of finding a global optimum.

In each iteration, the algorithm looks for the best available productivity (highest value) for the current employee (algorithm 1) or current position (algorithm 2).

Once an assignment is made, e.g., employee 0 gets position 1, the algorithm never re-evaluates that choice, even if that position would have been much more productive for a later employee.

In order to find a global optimum, it follows a "best-at-the-moment", i.e., iterating through the data and keeping updating a current maximum or minimum, rule to build the solution step-by-step.

Do you think the problem would allow for a divide-and-conquer solution? Why?

Theoretically, Divide-and-Conquer requires that the problem be broken into independent subproblems.

On one hand, if we implement D&C algorithm by division, if we divide the set of employees by half and solve them independently, both subsets might "claim" the same high-productivity positions, i.e., merging these solutions would require re-solving the problem, which is redundant.

On the other hand, if we implement D&C algorithm by subtraction, the set of employees will decrease by one each call, and we would have obtained the same result at the end.

However, I would say that a Divide-and-Conquer (D&C) approach is the most suitable strategy for this specific problem.

Algorithmics	Student information	Date	Number of session
	UO: 307423	19/03	4
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Does the ASSIGNMENT proposed by the CEO and currently implemented achieve the optimal solution (the highest possible total productivity)? Why?

No, it does not guarantee an optimal solution.

Greedy algorithms can often come off as short-sighted due to the fact that they fail to find the global optimum for assignment problems.

For example, if employee 0 is assigned their best position, that position might have been the only one where employee 3 was productive, which may cause the total sum of productivity to suffer.

To find the true maximum productivity, Backtracking is one of the approaches that should be taken in order to explore all combinations.